SMART RESTAURANT MANAGEMENT SYSTEM

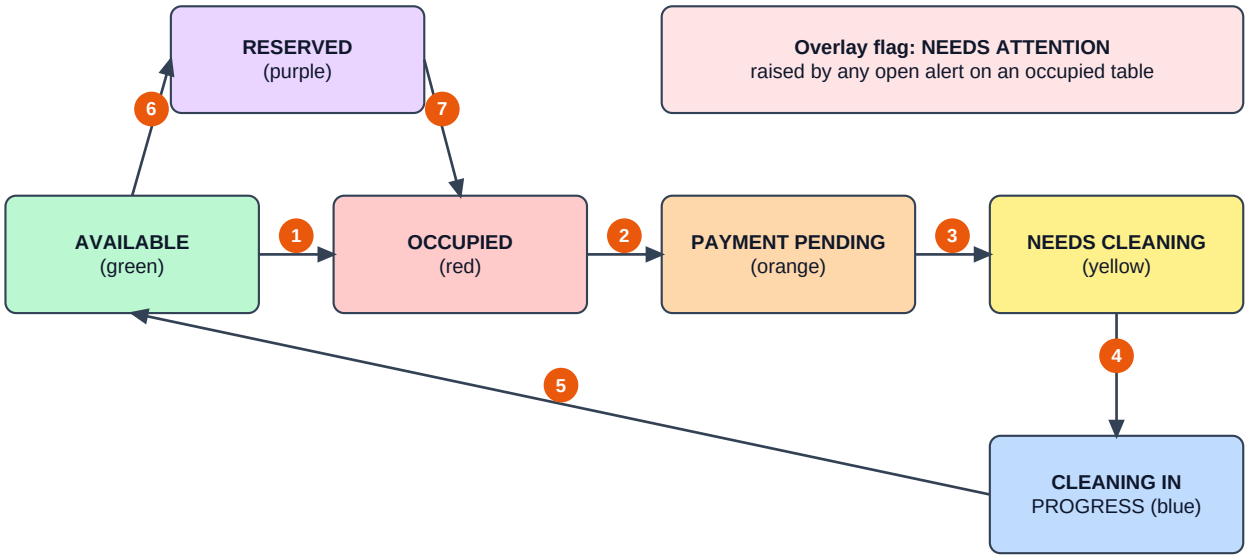
Workflows and State Machines

Table states, order lifecycle, alert rules and process flows

DOCUMENT ID	SRMS-06
VERSION	1.0
DATE	06 November 2026
STATUS	Draft for review
PHASE	Phase 1: event-driven core | Phase 2: AI camera detection
PURPOSE	The behavioural heart of the system: how table status changes, when alerts fire, and how each process flows.

1. Table State Machine

Table status is owned by the **state engine** in the backend. No client can write a status directly; clients send events (scan, order placed, payment received, cleaning done) and the engine decides the transition. Every transition is logged.



#	Transition	Trigger event	Guard / side effect
1	Available to Occupied	Guest scans QR, or staff seats a walk-in, or Phase 2 detects guests seated	Creates table session; starts timer
2	Occupied to Payment Pending	Guest requests bill or staff generates bill	Locks bill total; raises Request Bill alert
3	Payment Pending to Needs Cleaning	Bill fully paid (gateway webhook or cash)	Closes session; creates cleaning task
4	Needs Cleaning to Cleaning In Progress	Cleaner taps Start	Records start time
5	Cleaning In Progress to Available	Cleaner taps Done	Records duration; clears flags; checks waitlist
6	Available to Reserved	Reservation window starts (for example 15 minutes before)	Cleaning priority increases before this
7	Reserved to Occupied	Reserved guest arrives and is seated	Links reservation to session

Other transitions: any state to **Out of Service** by cleaner/manager with a note, and back to Needs Cleaning when restored. Manager override can set any valid state with a mandatory reason. Occupied to Needs Cleaning is allowed if guests walk out (close reason WALK_OUT) and the manager closes the session.

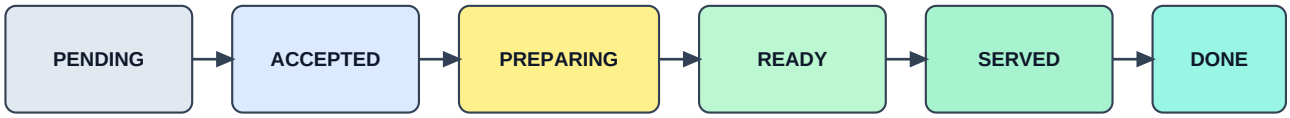
2. Floor Map Colour and Icon Legend

State / flag	Colour	Icon / extra info on tile
Available	Green	Seats count
Occupied	Red	Guest count, session timer
Payment Pending	Orange	Bill amount, payment icon
Needs Cleaning	Yellow	Time waiting for cleaner
Cleaning In Progress	Blue	Cleaner name, timer

State / flag	Colour	Icon / extra info on tile
Reserved	Purple	Reservation time and name
Out of Service	Grey	Reason
Needs Attention (overlay)	Pulsing ring or bell icon on any tile	Alert type, age of alert

Accessibility: colour is never the only signal. Each state also has a text label and icon so colour-blind staff can read the map.

3. Order and Item Lifecycle



- Order status is derived from item statuses (for example Ready when all active items are Ready).
- Cancellation: Pending and Accepted items can be cancelled by guest or staff; Preparing or later needs manager approval and records waste.
- An item marked out of stock by the kitchen is cancelled with reason and the guest is notified with an option to replace it.

4. Automatic Alert Rules

All thresholds are defaults and editable in admin settings.

Alert	Condition (default)	First recipient	Escalation
Call Waiter	Guest taps Call Waiter	Assigned waiter	Manager after 90 seconds unacknowledged
Request Bill	Guest taps Request Bill	Assigned waiter	Manager after 120 seconds
Long Wait (no order)	Table Occupied for 10 minutes with no order	Assigned waiter	Manager after 5 more minutes
Order Delayed	Any item exceeds its prep time by 5 minutes	Kitchen and waiter	Manager after 10 minutes
Unattended Table	Orders served but no staff interaction for 25 minutes	Assigned waiter	Manager
Payment Failed	Gateway reports failure	Assigned waiter	None; guest may retry or pay cash
Long Stay	Session longer than 90 minutes while waitlist is not empty	Manager	None
Cleaning Delay	Needs Cleaning for more than 5 minutes	Cleaners and manager	Manager after 10 minutes
AI Suggestion (Phase 2)	Camera detects table cleared or guests waiting	Waiter or manager	Confirm or dismiss

5. Key Process Flows

5.1 Guest scans and orders

1. Guest scans QR; browser opens the PWA with the table token.
2. Backend validates the token, finds or creates the active session, issues a session token, sets table Occupied (event logged).

3. Guest fills the shared cart; the server prices it; guest taps Place Order.
4. Backend saves the order and items in one transaction, then publishes events: kitchen ticket, waiter notification, floor map update.
5. Kitchen updates item status; events push live status to the guest and the map.

5.2 Payment

1. Guest taps Pay; backend creates a payment record with an idempotency key and a gateway order.
2. Guest completes UPI or card payment in the gateway sheet.
3. Gateway calls the webhook; backend verifies the signature and marks the payment SUCCESS.
4. If the bill is fully paid: session closes, table moves to Needs Cleaning, cleaning task is created, invoice is issued.
5. Guest receives a receipt. The client never decides success on its own; it polls or listens for the server confirmation.

5.3 Cleaning

1. Cleaning task appears in the cleaner queue with a push sound.
2. Cleaner taps Start (table turns blue), cleans, taps Done.
3. Table becomes Available (green). The waitlist is checked and the next party is notified if any.

5.4 Alert handling

1. Alert is raised (guest, system rule or AI) and the table gets the Needs Attention flag.
2. Assigned waiter is notified. Acknowledge shows who is handling it.
3. Waiter resolves after serving the guest; flag clears when no open alerts remain.
4. If unacknowledged past the threshold, the alert escalates to the manager and is highlighted.

6. Edge Cases and Rules

Case	Handling
Guest scans the wrong table QR	Show table label prominently at the top with a Change table option; staff can move the session
Several guests scan the same table	All join the same session and cart; each gets the same live status
Guest scans after paying and leaving	Session closed, so a new session is started only if the table is Available or staff reopen it; prevents ordering to a dirty table
Guest leaves without paying	Manager closes session with reason WALK_OUT; bill stays unpaid and is reported; table goes to Needs Cleaning
Payment succeeded but guest closed the browser	Webhook still completes payment and closes the bill
Duplicate webhook or double tap	Idempotency keys and unique provider payment IDs ignore duplicates
Internet drops in kitchen	Local retry with banner; tickets resync on reconnect; waiter can announce by voice; no orders lost
Item price changed during session	Order uses the price at order time (snapshot); new orders use the new price
Table moved or merged for a big group	Manager merges sessions; floor map shows linked tables
Manager override conflicts with AI event (Phase 2)	Manual override wins and AI events are ignored for a cooldown period